

Driver Drowsiness Detection — Implementation - Part 1

Project Title	Driver Drowsiness Detection System
Team Number	25
Team Members	Vangara Sridhar - 2023BCS0106 Suragouni Sahithi - 2023BCS0070 Vatte Ramya Sri - 2023BCS0208 Abhay Devanand Sawale - 2023BCS0046
Scope	Dataset collection, preprocessing, and approximately 25% methodology implementation
Date	17 September 2026

1. Project Overview

This project implements an end-to-end computer vision system for detecting driver drowsiness from webcam frames. The pipeline is camera capture -> Next.js frontend -> FastAPI backend -> YOLO classification -> JSON prediction -> visual status and alarm feedback.

Task 4 completion status

Requirement	Evidence	Status
Dataset collection	UTA-RLDD downloaded/configured through KaggleHub; documented in DATASET_INFO.md.	Completed
Pre-processing	Class mapping, reproducible split, YOLO directory generation, OpenCV decoding and RGB conversion.	Completed
Approx. 25% methodology	Webcam capture, upload, API routing, model loading, inference parsing, display and temporal alarm.	Implemented
Remaining work	Evaluation, direct eye-closure measurement, calibration, deployment hardening and final documentation.	Listed

2. Dataset Collection and Preparation

The project uses the UTA Real-Life Drowsiness Dataset (UTA-RLDD) from Kaggle. Source folders active and fatigue are mapped to Alert and Drowsy for the application.

Item	Recorded value
Total images	9,054
Distribution	Alert: 5,862 (~65%); Drowsy: 3,192 (~35%)
Generated split	Train: 6,337 Validation: 1,358 Test: 1,359
Class weights	Alert: 0.77 Drowsy: 1.42

Pre-processing implemented

Dataset discovery checks the KaggleHub cache and configured data directory. Image paths and labels are collected from class folders. Samples are split reproducibly with a fixed seed using a 70/15/15 ratio. A YOLO classification

directory is generated. Uploaded images are validated and decoded with OpenCV, converted from BGR to RGB, and passed to YOLO.

The source dataset is video-frame based. A future experiment should split by participant/video to reduce near-duplicate leakage between train and test sets.

3. Implemented Methodology

Stage	Module	Output
Capture	Camera.jsx	JPEG webcam frame
Transport	lib/api.js	multipart/form-data /predict
Decode	image_processing.py	Validated OpenCV image
Inference	model.py + inference.py	Class and confidence
Feedback	page.jsx + DetectionResult.jsx	Status, streak and alarm

Alarm behavior

Live monitoring sends approximately one frame per second. When the model returns Drowsy continuously, the frontend records the start time and activates the alarm after five seconds. An audio file can be placed at frontend/public/alarm.mp3; a browser tone is used as fallback. Alert results reset the timer.

Model safeguard

The backend validates checkpoint class names and rejects ImageNet-style models that lack Drowsy, Alert or Non_Drowsy. This prevents the earlier jersey/sunglasses failure mode.

Verification

Check	Result
npm run lint	Passed - no ESLint warnings
npm run build	Passed - Next.js build succeeded
Backend compile/import	Passed
GET /health	Passed - model_loaded=true
POST /predict	Passed - drowsiness-compatible response
CORS local-origin test	Passed

4. Major Tasks Remaining

Priority	Module	Next action
High	Formal evaluation	Report accuracy, precision, recall, F1, confusion matrix and calibration.
High	Eye-closure duration	Add face/eye landmarks or an eye-state model to measure closed eyes directly.
High	Reproducibility	Record dataset version, training command, checkpoint provenance and participant-level split.
Medium	Alarm UX	Add alarm.mp3 and test browser user-gesture/autoplay behavior.

Medium	Deployment	Configure production origins, Render/Vercel variables, logging and cold-start messaging.
Low	Analytics	Add session export, timestamps and privacy-preserving metrics.

5. Conclusion

Task 4 Part 1 is substantially implemented. Dataset integration and preprocessing are documented, and the first functional quarter of the methodology is demonstrated through the working webcam-to-inference pipeline. The next milestone is formal evaluation and a dedicated eye-closure/temporal model.